- This lab will cover Stacks & Queues.
- When approaching the problems, think before you code. Doing so is good practice and can help you lay out possible solutions.
- Think of any possible test cases that can potentially cause your solution to fail!
- If you finish early, you may leave early after showing the TA your work. Ory you may stay and help other students. If you don't finish by the end of the lab, we recommend you complete it on your own time. Ideally you should not spend more time than suggested for each problem.
- Your TAs are available to answer questions in lab, during office hours, and on Edstem.

Vitamins (40 minutes)

---

1. What is the output of the following code? Draw the final stack(s) or queue after all lines have executed (15 minutes)

a)
```
s = ArrayStack()
t = ArrayStack()

for val in [1, 2, 3, 4]:
      s.push(val)

for i in range(2):
      t.push(s.pop())

s.push(t.pop())
print(s.pop())
print(t.pop())
print(s.top())
```

b)
```
q = ArrayQueue()

i = 2
q.enqueue(1)
q.enqueue(2)
q.enqueue(4)
q.enqueue(8)
i += q.first()
q.enqueue(i)
q.dequeue()
q.dequeue()
print(i)
print(q.first())
```

c)
```
q = ArrayQueue()

item = [2]
q.enqueue(item)
q.enqueue([7])
alias = q
saved = q.dequeue()
saved.append(4)
q.enqueue(saved)
print(q.first())
alias.first().append(9)
print(q.dequeue())
print(item)
print(len(alias))
```

2. A student changes the `pop` method of `ArrayStack` to the implementation below. Assume `s` contains `[10, 20, 30]` from bottom to top. (5 minutes)

```
def pop(self):
    if self.is_empty():
        raise Exception("Stack is empty")

    item = self.data[-2]
    self.data.pop()
    return item
```

What is the output of the following code?

```
s = ArrayStack()
s.push(10)
s.push(20)
s.push(30)
x = s.pop()
print(x)
print(s.top())
print(len(s))
```

3. Trace the following functions. Describe what the function does, and give a meaningful name to the function: (15 minutes)

a)
```
def mystery(s):
    if len(s) == 1:
```

```
        return s.top()
      else:
        val = s.pop()
        result = mystery(s)

        if val < result:
             result = val
        s.push(val)
        return result
```

b)
```
def mystery(q):
    if q.is_empty():
        return
    else:
        val = q.dequeue()
        mystery(q)
        if val % 2 != 0:
           q.enqueue(val)
```

c)
```
def mystery(input_str):
    s = ArrayStack()
    q = ArrayQueue()
    for char in input_str:
        s.push(char)
        q.enqueue(char)
    while not s.is_empty():
        if s.pop() != q.dequeue():
            return False
    return True
```

4. Fill out the prefix, infix, postfix table below: (20 minutes)

Prefix	Infix	Postfix	Value
- * 4 5 + 3 2			
	(18 - 6) / (2 + 1)		
		8 3 2 * + 5 -	
* + 2 3 - 9 5			

Coding (75 minutes)

In this section, it is strongly recommended that you solve the problem on paper before writing code. Note that you should not access the underlying list in the ArrayStack implementation. **Treat it as a black box and only use the len, is_empty, push, top, and pop methods.**

Note:

I. Download the **ArrayStack.py**, **ArrayList.py & ArrayQueue.py** files under Content/Labs on Brightspace

II. Import the classes like this: `from ArrayStack import *`

1. You are given a list of prices where prices[i] is the price of the ith item in a shop.

   There is a special discount for items in the shop. If you buy the ith item, then you will receive a discount equivalent to prices[j] where j is the minimum index such that j > i and prices[j] <= prices[i]. Otherwise, you will not receive any discount at all.

   Return a list answer where answer[i] is the final price you will pay for the ith item of the shop, considering the special discount

   ex) Input: prices = [8,4,6,2,3]
   Output = [4,2,4,2,3]

   ```
   def finalPrices(prices):
       """
       : input_str type: list
       : return type: list
       """
   ```

2. The town of Dingle gives every citizen a ticket with a number from 1 to 100 (numbers may repeat) and lines everyone up in a single row. The mayor may remove any **two adjacen**t citizens whose ticket numbers add up to an **even** value. When a pair is removed, the line closes up, and the people on **either side of the removed pair become neighbors.** If their tickets now add up to an even value, they can be removed as well. He keeps removing pairs until no two adjacent citizens have an even sum, and everyone left in line wins 100 gold pieces.

   Write an **iterative function** that returns the **smallest number of winners** possible. You are **only allowed to use one ArrayStack** as an additional data structure. Do not use any helper functions or change the function signature. (20 minutes)

   The **runtime** of the function should be **linear**.

ex) tickets =
lucky_draw(tickets) returns 10
No adjacent pair adds up to an even value, so no one is removed.

ex) tickets = [3, 8, 6, 5, 2]
lucky_draw(tickets) returns 1
Removing (8, 6) leaves [3, 5, 2]. 3 and 5 are now neighbors, so removing (3, 5) leaves [2]

ex) tickets = [7, 1, 3, 6, 2, 1, 2, 4, 8]
lucky_draw(tickets) returns 1
Removing (7, 1), (6, 2), and (2, 4) leaves [3, 1, 8]. 3 and 1 are now neighbors, so removing (3, 1) leaves [8]. .

.

```
def lucky_draw(tickets):
        """
        : tickets type: list
        : return type: int
        """
```

3. Create an **iterative function** that evaluates a valid prefix string expression. You may only use **one ArrayStack** as an additional data structure to the given setup. Do not use any helper functions or change the function signature.

   In addition, each character is separated by one white space and numbers may have more than one digit. Therefore, we will use the split function to create a new list of each substring of the string separated by a white space. You may assume all numbers will be positive. (30 minutes)

ex) exp_str is " - + * 16 5 * 8 4 20"
exp_lst = exp.split(" ") → ["-", "+", "*", "16", "5", "*", "8", "4", "20"]
eval_prefix(exp_str) returns = 92

```
def eval_prefix(exp_str):
    """
    : exp type: str
    : return type: int
    """
    exp_lst = exp_str.split( )
```

Hint:
To check if a string contains digits, use `.isdigit( )`.
To check if a string is an operator, you may want to do `if char in "-+/*"` similarly to how you checked for vowels.

4. Implement reverse_prefix(q, k). Reverse the first k elements of q while leaving the remaining elements in their original order. Mutate q and return None. Assume 0 <= k <= len(q). (20 minutes)

```
# q initially contains [10, 20, 30, 40, 50]
reverse_prefix(q, 3)
# q now contains [30, 20, 10, 40, 50]
```

You can use one additional data structure of space O(n). Access q only through enqueue, dequeue, first, is_empty, and len. Do not use recursion, lists, or queue internals. Use O(n) total time.

```
def reverse_prefix(q, k):
    pass
```

5. Implement RecentCounter. Its ping(t) method records a request at integer time t and returns the number of recorded requests in the inclusive interval [t - x, t], x being a value imputed while initializing RecentCounter. Call times are strictly increasing. Use one ArrayQueue and O(1) extra scalar storage. Keep only requests in the current interval and do not scan the whole queue on every call. (20 minutes)

```
counter = RecentCounter(3000)
print(counter.ping(10))    # 1
print(counter.ping(20))    # 2
print(counter.ping(3010))  # 3
print(counter.ping(3011))  # 3
print(counter.ping(7000))  # 1
```

For m >= 1 calls starting from an empty counter, require Θ(m) total time under the queue's amortized bounds. State the worst-case number of requests one call may remove. A request exactly 3000 time units old must still be counted.

```
class RecentCounter:
    def __init__(self, x):
        self.data = ArrayQueue()
        self.interval = x

    def ping(self, t):
        pass
```

6. Implement ticket_time(tickets, k). Person i initially needs tickets[i] tickets, where every entry is a positive integer and 0 <= k < len(tickets). People start in index order. Each turn takes one second: the front person buys one ticket, then rejoins the back if they still need more. Return the elapsed time when the person originally at index k finishes.

   Do not modify tickets. Simulate the process with one ArrayQueue storing each person's original index and remaining demand as a pair. Apart from this queue and its pairs, use O(1) scalar storage. Access the queue only through its public methods. Stop as soon as the target person finishes.

   ```
   ticket_time([2, 1, 3], 0)  # 4
   ticket_time([1, 2, 1], 1)  # 4
   ticket_time([3], 0)        # 3
   ```

   Use O(n + T) total time under the queue's amortized bounds and O(n) auxiliary space, where n is the number of people and T is the returned elapsed time.

   ```
   def ticket_time(tickets, k):
       pass
   ```